

Unificação

João Eduardo Montandon
DCC024

A essência do Prolog está no algoritmo de **Unificação**.

- Entrada ➡ Dois termos a serem unificados: esquerda e direita.
- Objetivo ➡ Encontrar "caminho" que torne os termos equivalentes.

Como? 🤔

Substituição

Uma **substituição** é uma função que mapeia variáveis e termos.

$$\sigma = \{X \rightarrow a, Y \rightarrow f(a, b)\}$$

- A substituição σ mapeou X para a e Y para $f(a, b)$.
 - O resultado da substituição é uma *instância* do termo original.
-

```
?- append([1,2],[3,4],Z).  
Z = [1, 2, 3, 4].
```

$$\underset{t_1}{append([1, 2], [3, 4], Z)} \equiv_{\{Z \rightarrow [1, 2, 3, 4]\}} \underset{\sigma}{append([1, 2], [3, 4], [1, 2, 3, 4])} \underset{t_2}{}$$

Dois termos são *unificáveis* se existir alguma substituição σ que os tornem equivalentes.

Exemplo: quais termos são unificáveis?

1. $a = b.$
2. $f(X, b) = f(a, Y).$
3. $f(X, b) = g(X, b).$
4. $a(X, X, b) = a(b, X, X).$
5. $a(X, X, b) = a(c, X, X).$
6. $a(X, f) = a(X, f).$

-
1. Não.
 2. $\sigma \{X \rightarrow a, Y \rightarrow b\}.$
 3. Não.
 4. $\sigma \{X \rightarrow b\}.$
 5. Não.
 6. $\sigma \{\}$

Dois termos podem ser unificados por diferentes substituições.

```
parent(fred, mary).  
parent(X,Y) = parent(fred, Y).
```

- $\sigma_1 \{X \rightarrow fred\}$
- $\sigma_2 \{X \rightarrow fred, Y \rightarrow mary\}$

Qual é melhor? 🤖

Most General Unifier (MGU)

A escolha se dará pelo σ com **menor número de substituições**, pois ele leva a um termo **mais genérico**.

`parent(fred, Y)` é mais genérico que `parent(fred, mary)`

Unificação Em Todo Lugar

Unificação é a base para computação de vários comandos em Prolog.

```
?- reverse([1,2,3], X).  
X = [3,2,1].
```

```
?- x = 0.  
X = 0.
```

```
?- [1,2,3] = [X|Y].  
X = 1, Y = [2,3].
```

Verificação De Ocorrências

Qual seria o valor de `X` para as consultas abaixo? 🤔

- `X = f(X)`
- `append([], X, [a|X])`

Algumas implementações não unificam termos que utilizam mesma variável.

Modelo De Execução

Modelo utilizado pelo interpretador para realizar computações.

1. Resolution.

2. Solve.

Resolution

- Realiza unificação entre uma cláusula do programa e os termos da consulta realizada.
 - Cada resolução representa um passo no processo de prova.
-

```
fun resolution(clause, goals)
    sub = MGU(head(clause), head(goals))
    return sub(tail(clause) @ tail(goals))
```

- `clause`: Lista de termos da cláusula do programa.
 - `goals`: Lista de termos da prova.
 - `head()` e `tail()`: cabeça e cauda de uma lista qualquer.
-

Exemplo

- `goals`: `[p(X), s(X)]`
- `regra`: `p(f(Y)) :- q(Y), r(Y).`

```
resolution([p(f(Y)), q(Y), r(Y)], [p(X), s(X)])
    sub = MGU(p(f(Y)), p(X)) ⇒
           {X → f(Y)}
    sub([q(Y), r(Y), s(X)]) ⇒
           [q(Y), r(Y), s(f(Y))]
```

Solve

- Procedimento utilizado pelo interpretador para realizar uma prova.
 - Entrada ➡ Lista de termos
 - Objetivo ➡ Reduzir a lista através da aplicação sucessiva de `resolution()`
 - Se ao final lista estiver vazia, prova foi bem sucedida.
-

```
fun solve(goals)
    if goals = []
        return true
    else
        foreach (clause c : program)
            if(head(c) unifies with head(goals))
                solve(resolution(c, goals))
        return false
```

Exemplo

```
p(f(Y)) :- q(Y), r(Y).
q(g(Z)).
q(h(Z)).
r(h(a)).
```

```
?- p(X).
```

Passo 1

```
solve([**p(X)**])
    solve([q(Y), r(Y)])
    false
    false
    false
```

Passo 02

```
solve([**p(X)**])
  solve([**q(Y)**, r(Y)])
    false
    solve([r(g(Z))])
    solve([r(h(Z))])
    false
  false
false
false
false
```

Passo 03

```
solve([**p(X)**])
  solve([**q(Y)**, r(Y)])
    false
    solve([**r(g(Z))**])
      false
      false
      false
      false
    solve([r(h(Z))])
    false
  false
false
false
false
```

Passo 04

```

solve([**p(X)**])
  solve([**q(Y)**, r(Y)])
    false
    solve([r(g(Z))])
      false
      false
      false
      false
      solve([**r(h(Z))**])
        false
        false
        false
        solve([****])
          true
      false
  false
false

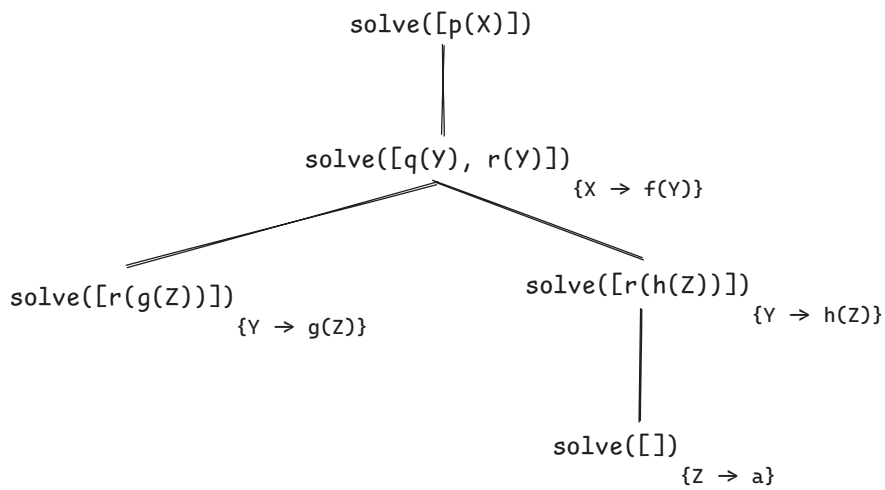
```

Que estrutura é gerada por essa computação? 🤔

Árvores De Prova

Árvore gerada durante a realização da prova de uma consulta em Prolog.

- Dois tipos de nós: *nothing* e *solve*.
 - Todo nó *nothing* é uma folha e pode ser omitida.
 - Cada nó *solve* contém uma lista de termos.
 - Se a lista for vazia, é uma folha.
 - O nó raiz contém a consulta original.
-



Semântica

Em que ordem essa árvore é gerada? 🤔

programa1.pl

```
p :- p.  
p.
```

programa2.pl

```
p.  
p :- p.
```

Qual árvore é gerada para cada programa se consultarmos `?- p.` ?

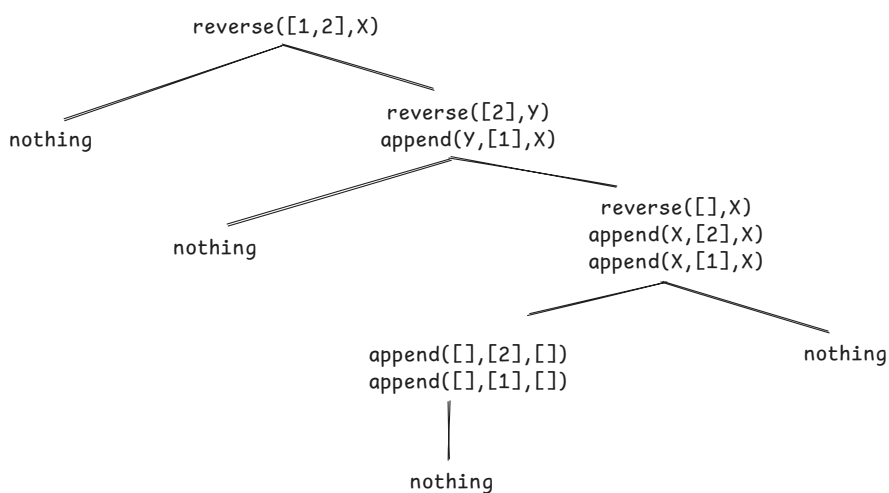
Em Prolog, árvores são geradas em profundidade, da esquerda para direita (conforme ordem das cláusulas).

Um Problema De Nome

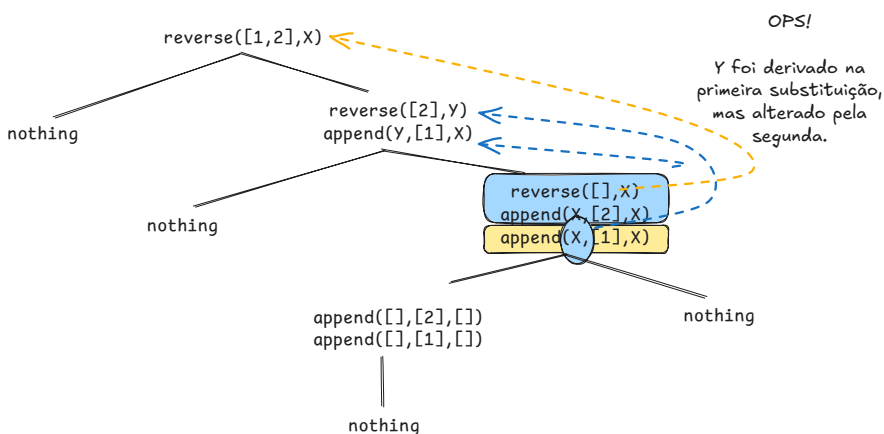
```
reverse([], []).  
reverse([H|T], X) :- reverse(T,Y), append(Y, [H], X).
```

```
?- reverse([1,2], X).
```

Construa a árvore de prova.

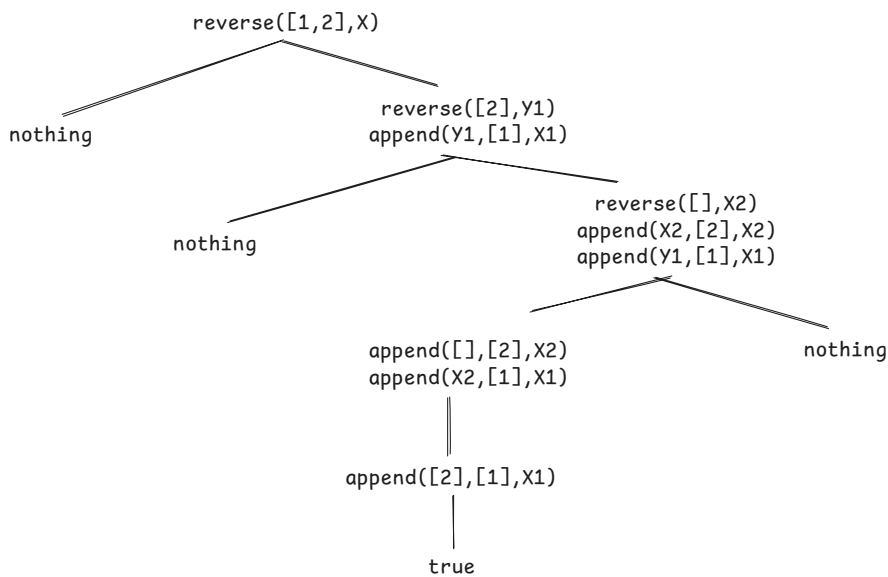


Por que deu errado? 🤔



- O que presenciamos? uma captura de nome.

- Mesma variável em diferentes cláusulas.
- Como resolver? Redução α das cláusulas em cada chamada.



- Necessária para preservar o escopo de variáveis.
 - O escopo de `X` é sua cláusula.
- Renomeie toda vez que uma cláusula é usada.

Algumas Funções Úteis

1. Strings
2. Entrada e Saída
3. `assert` e `retract`
4. Comando de corte (`!`)

Strings

- Representadas por aspas simples (`'Hello world'`)
- Mesmo comportamento de um literal, porém permite barra de espaço.

```
?- 'eh pai'('joao eduardo', gabriel) = 'eh pai'(Y,Z).  
Y = 'joao eduardo',  
Z = gabriel.
```

Entrada E Saída

- `write(X)`: escreve `X` na tela.
- `read(X)`: armazena a leitura em `X`.
- `nl` $\equiv$ `write('\n')`.

⚠ Como tudo em Prolog, são termos a serem provados, **porém produzem um efeito colateral!**

```
?- write('Hello, world!').  
Hello, world!  
true.  
  
?- read(X).  
|: hello.  
X = hello.
```

Depuração

`write` pode ser útil especialmente para depurar provas.

```
p :- append(X, Y, [1,2]),  
      write('X='), write(X), write('\tY='), write(Y), nl,  
      X=Y.
```

```
?- p.  
X=[]      Y=[1,2]
```

```
X=[1]    Y=[2]
X=[1,2]  Y=[]
false.
```

assert / **retract**

Permite adicionar e remover fatos/regras ao final do programa. **Use com moderação!**

```
?- parent(joe,mary).
false.

?- assert(parent(joe, mary)).
true.

?- parent(joe,mary).
true.
```

```
?- parent(joe,mary).
true.

?- retract(parent(joe, mary)).
true.

?- parent(joe, mary).
false.
```

Comando De Corte

- Interrompe prova caso cláusula onde foi declarada obtenha sucesso.
- Representado por `!`.

```
p :- member(X, [a,b,c]), write(X), !.  
p :- write(d).
```

```
?- p.  
a  
true.
```

Alguém Gosta De RPG?

```
/*  
  This is a little adventure game.  There are three  
  entities: you, a treasure, and an ogre.  There are  
  six places: a valley, a path, a cliff, a fork, a maze,  
  and a mountaintop.  Your goal is to get the treasure  
  without being killed first.  
*/  
  
/*  
  First, text descriptions of all the places in  
  the game.  
*/  
description(valley,  
  'You are in a pleasant valley, with a trail ahead.').  
description(path,  
  'You are on a path, with ravines on both sides.').  
description(cliff,  
  'You are teetering on the edge of a cliff.').  
description(fork,  
  'You are at a fork in the path.').  
description(maze(_),  
  'You are in a maze of twisty trails, all alike.').  
description(mountaintop,  
  'You are on the mountaintop.').  
  
/*  
  report prints the description of your current  
  location.  
*/  
report :- at(you,X),  
  description(X,Y),
```

```

write(Y), nl.

/*
  These connect predicates establish the map.
  The meaning of connect(X,Dir,Y) is that if you
  are at X and you move in direction Dir, you
  get to Y.  Recognized directions are
  forward, right, and left.
*/
connect(valley,forward,path).
connect(path,right,cliff).
connect(path,left,cliff).
connect(path,forward,fork).
connect(fork,left,maze(0)).
connect(fork,right,mountaintop).
connect(maze(0),left,maze(1)).
connect(maze(0),right,maze(3)).
connect(maze(1),left,maze(0)).
connect(maze(1),right,maze(2)).
connect(maze(2),left,fork).
connect(maze(2),right,maze(0)).
connect(maze(3),left,maze(0)).
connect(maze(3),right,maze(3)).

/*
  move(Dir) moves you in direction Dir, then
  prints the description of your new location.
*/
move(Dir) :- at(you,Loc),
             connect(Loc,Dir,Next),
             retract(at(you,Loc)),
             assert(at(you,Next)),
             report,
             !.

/*
  But if the argument was not a legal direction,
  print an error message and don't move.
*/
move(_) :- write('That is not a legal move.\n'),
           report.

/*
  Shorthand for moves.
*/

```

```
forward :- move(forward).
left :move(left).
right :move(right).

/*
    If you and the ogre are at the same place, it
    kills you.
*/
ogre :- at(ogre,Loc),
        at(you,Loc),
        write('An ogre sucks your brain out through\n'),
        write('your eye sockets, and you die.\n'),
        retract(at(you,Loc)),
        assert(at(you,done)),
        !.

/*
    But if you and the ogre are not in the same place,
    nothing happens.
*/
ogre.

/*
    If you and the treasure are at the same place, you
    win.
*/
treasure :- at(treasure,Loc),
            at(you,Loc),
            write('There is a treasure here.\n'),
            write('Congratulations, you win!\n'),
            retract(at(you,Loc)),
            assert(at(you,done)),
            !.

/*
    But if you and the treasure are not in the same
    place, nothing happens.
*/
treasure.

/*
    If you are at the cliff, you fall off and die.
*/
cliff :- at(you,cliff),
        write('You fall off and die.\n'),
        retract(at(you,cliff)),
```

```

    assert(at(you,done)),
    !.
/*
    But if you are not at the cliff nothing happens.
*/
cliff.

/*
    Main loop.  Stop if player won or lost.
*/
main :- at(you,done),
    write('Thanks for playing.\n'),
    !.
/*
    Main loop.  Not done, so get a move from the user
    and make it.  Then run all our special behaviors.
    Then repeat.
*/
main :- write('\nNext move -- '),
    read(Move),
    call(Move),
    ogre,
    treasure,
    cliff,
    main.

/*
    This is the starting point for the game.  We
    assert the initial conditions, print an initial
    report, then start the main loop.
*/
go :- retractall(at(_,_)), % clean up from previous runs
    assert(at(you,valley)),
    assert(at(ogre,maze(3))),
    assert(at(treasure,mountaintop)),
    write('This is an adventure game. \n'),
    write('Legal moves are left, right, or forward.\n'),
    write('End each move with a period.\n\n'),
    report,
    main.

```

```

?- go.

```


Enjoy! 😊
